

Reactions and the partition: opt-in eventual consistency in actor-native systems

Alvaro Rivera

2026-05-12

Abstract

This paper makes a design theory contribution in the sense of Hevner, March, Park, and Ram (2004): it identifies a new design construct — the developer-controlled pragmatic partition of an event’s implied work into now and deferred — derives the design principles required for that partition to be exercised in practice, and presents an instantiation in which those principles have been realized in production. Eventual consistency is treated, in the CQRS literature, as a defining commitment of the architecture: separated read and write stores synchronize asynchronously, and applications must absorb the contract. This paper argues for actor-native systems that the contract is upside-down. The primitive is not eventual consistency; it is the developer’s pragmatic partition of the work an event implies into a *now* branch (what the verb’s contract promises before responding) and a *deferred* branch (placed wherever the developer chose — same thread, another node, the cluster). Reactions are the artifact through which that partition is exercised, and they are not async events under another name: each Reaction matches a *trajectory* through the actor’s journal, statically compiled against the domain’s libraries, with typed identifiers propagated across stages. Reactions serve a double purpose — pragmatic deferral and categorical segregation of operational concerns away from domain methods — and the segregation is structural, not stylistic: the language itself blocks the operational concern from leaking back into the actor’s state. Both purposes share a single guardian — friction — and collapse together if it is high. Three consequences fall out of the primitive when its exercise is low-friction: a domain library that closes (the primary consequence — declarable as “task done” because it is free of present and future operational tools), fast verbs by construction, and opt-in eventual consistency (the developer names each deferred effect by hand, inverting the CQRS-classical assumption). The argument is conceptual; code references throughout point to the public Puppeteer codebase, in which the Reaction primitive — its two modes (*Cue* and *Job*), its three action planes (*Program.Emit*, *Causation.Continue*, *Metadata.Elide*), its activation scopes, and its multi-event pattern matching — instantiates the primitive concretely.

Reactions and the partition

TL;DR

When CQRS introduces eventual consistency, the standard treatment is as a cost: a contract between separated read and write stores that the application must absorb. This paper argues that for actor-native systems the contract is upside-down. The primitive is not eventual consistency at all; it is the developer’s pragmatic partition of the work an event implies into *now* (what the verb’s contract promises before responding) and *deferred* (placed wherever the developer chose — same thread, another node, the cluster).

Reactions are the artifact through which the partition is exercised — and they are not async events under another name. Each Reaction matches a *trajectory* through the actor’s journal, a sequence of one or more stages naming a saga or lifecycle, statically compiled against the domain’s libraries with typed identifiers propagated across stages. Reactions serve two purposes simultaneously. The first is **pragmatic deferral** — for latency budget, parallelization, scaling. The second is **categorical segregation** — keeping operational concerns (email, BI, third-party messaging, telemetry) from propagating into domain methods. Reactions are *not* technology-free; they are precisely where that technology lives, **encapsulated** so that other domain verbs do not inherit the dependency, and the encapsulation is structural — the language itself blocks the operational concern from leaking back. The library, in turn, stays free of that propagation, built under its legitimate programming paradigm (currently object orientation). Both purposes share one guardian: **friction**. If Reactions cost the developer more than inline code, the partition is not exercised; verbs degrade and the domain library is contaminated by precipitation. The friction is an architectural responsibility of the framework, not a discipline question for the developer.

Three consequences fall out of the primitive when it is low-friction. The primary one is a **domain library that closes** — that can be declared *task done* because it is structurally free of present and future operational tools. The other two are **fast verbs by construction** and **opt-in eventual consistency** (the developer names each deferred effect by hand, inverting the CQRS-classical assumption). **Eventual consistency is the shadow of the partition, not the design choice.**

Claims this paper makes

1. **Logical, not physical, CQRS.** Within an actor, write and read paths operate on a single in-memory state. `PerformCmd` and `PerformQry` are verbs over the same store; there is no synchronization between separated write and read sides because there are no separated sides. The eventual-consistency contract that defines classical CQRS does not apply at the actor boundary. (*Verification: code references to `PerformCmd`/`PerformQry` over a shared actor state in §6.*)
2. **Within an event’s frame, formal correctness places everything in *now*.** The work an event implies — every domain effect entailed by the verb’s contract — should, formally, resolve before the event closes. The *now/deferred* split is therefore not a decomposition deduced from correctness; it is a pragmatic boundary the developer draws inside the event’s implied work. (“Everything” here scopes to the event’s domain-implied work; concerns extrinsic to the domain are addressed in claim 4.)
3. **The *now/deferred* partition is the primitive.** The developer chooses, feature by feature, what the verb’s response promises now and what is deferred. The choice is pragmatic: it is governed by latency budget, parallelization opportunity, and the categorical purity argument made in claim 4 — not by a deduction from the formal contract.
4. **The artifact realizing the partition serves a double purpose.** In the instantiation developed here, this artifact is the `Reaction`.
 - **(a) Pragmatic partition** — work that the verb cannot afford to consume within its latency budget, or that is more naturally executed in parallel, is deferred.
 - **(b) Categorical segregation** — extrinsic operational concerns (email, BI, third-party messaging, telemetry, webhooks) demanded by requirements but not part of the rich library land in the deferral artifact, not in domain methods. The segregation is of *propagation*, not of *presence*: the artifact may itself invoke email or telemetry technology, and that is appropriate — the technology is encapsulated where it can be exercised without leaking into other domain verbs. *The artifact is not technology-free; it is the encapsulation that keeps technology from propagating into the domain.* This extends the anti-porosity principle of Paper 1 to a fourth layer — the operational edge of the domain — and is compatible with the rich library being built under the legitimate programming paradigm (currently object orientation) that sustains the domain language. That paradigm and the conceptual primitives it permits (classes, inheritance, polymorphism, invariants, domain methods) are the legitimate tool *inside* the library (§3.1). *The encapsulation is structurally enforced, not conventional: in Puppeteer’s instantiation, a `Reaction`’s `emit`*

path runs as a query (PerformEmit with isQuery:true, blocking expose and journal writes) — the language refuses to let the operational concern leak back into the actor’s state. (Verification: ActorHandler.cs:1662–1733.)

5. **Deferred work carries a placement decision.** The developer, at the moment of deferral, also decides where the deferred work runs along a *proximity / urgency* axis: close-to-action (same thread, near-continuation, parent actor’s memory available) remote-and-deferred (another thread, another node, cluster-distributed). Placement is part of the primitive, not an implementation detail; it is orthogonal to whether the deferral is pragmatic (4a) or categorical (4b).
6. **Friction is the structural guardian of the partition.** The artifact realizing the partition must minimize the cost of declaring deferred work, or both purposes collapse simultaneously: if deferring costs the developer more lines or more cognitive load than inline code, the partition is not exercised, verbs degrade *and* the domain library is contaminated by precipitation. The contamination is the worse of the two failures because it is hard to revert and propagates couplings. Friction is an architectural responsibility of any complete instantiation, not a question of developer discipline. (*Verbatim author statement anchored as blockquote in §4.*)
7. **Three consequences of (3) + (4) + (6), ranked.** The primary consequence is **a domain library that closes** — declarable as *task done* by being free of present and future operational tools (§5.1, §10). Secondary consequences are **fast verbs by construction** (§5.2) and **opt-in eventual consistency**, in which the developer names each deferred effect by hand — the inverse of the CQRS-classical opt-out assumption (§5.3, honors the forward-reference of Paper 1 §6). The three are not features and not design choices; they fall out of the primitive when it is low-friction. *Fast verbs in particular are structural: a Reaction executes outside the write semaphore of PerformCommand; the verb returns to the caller without waiting for any deferred work to begin. (Verification: Reaction.cs:715–832, ActorHandler.cs:1662–1733.)*
8. **The continuation contract survives the deployment switch.** Deferred work in flight when a node hands off to its successor is not lost; red-black is orchestrated by the runtime — developers do not normally see the mechanism. The only common exception is coordinated cut-over of a Kafka producer/consumer pair when a message format changes, where both sides must release together. In Puppeteer’s instantiation, this property is realized by **Theater.aliveGate**; the mechanism itself is treated in detail in Paper 5 (zero-downtime deployment, forthcoming). The cross-actor continuity primitive that complements the in-actor continuity argued here is the subject of Paper 4, already published in the present series.
9. **Domain libraries become real artifacts.** In most enterprises today,

“the domain” does not exist as a library — it lives as classes scattered across each csproj, copied between services, drifting independently over time until “the customer model” or “the order model” has become a dozen incompatible structures across the codebase. The pattern of pure domain methods, preserved by Reactions encapsulating operational concerns (claim 4b), is the structural condition under which a single domain library, maintained as one artifact over years, becomes economically possible at all. *Reusability across endpoints is the operational form of closability* (claim 7).

10. **The artifact realizing the partition is declarative, multi-event pattern matching against the actor’s trajectory, statically compiled against the domain’s libraries.** It is not an event handler. It is a rule the engine applies over the actor’s journal: a sequence of one or more stages that, taken together, describe a trajectory through the actor’s history — a saga, a lifecycle, a recurring anomaly. Identifiers captured in one stage propagate to the next, correlating the events. The pattern is parsed and resolved against the domain’s libraries in build phase, not against text at runtime; a stage referring to a class the domain does not contain fails to compile. The propagation, the typed capture, and the static binding to the domain are jointly what distinguishes this primitive from event-driven continuations in Akka, grain reminders in Orleans, or topic consumers in Kafka. The instantiation that demonstrates this is the Reaction primitive of Puppeteer — its `Seek`, `ThenSeek`, and `ThenFinalSeek` stages are developed in §6.2 and §6.3 (*primary code anchors `Pattern.cs`, `Reaction.SolveActionReferences()` at `Reaction.cs:997`, `MatchTree.TryMatchAtLevel()`*).
11. **The proximity/urgency axis has two canonical points, not a continuous spectrum.** In Puppeteer’s instantiation, these are `Cue()` (push loop on a separate thread, operating against the actor’s live in-memory state, `ReadForward()` mandatory, sub-second latency) and `Job()` (on-demand against the replicated journal, no co-location with the live actor, freely placeable on a follower in another pod or another machine in the cluster). The two modes are structurally distinct, not points on a slider — they differ in source of state (memory vs journal), in trigger (push callback vs explicit invocation), and in what they presuppose about the host. The placement decision of claim 5 is not abstract: it is the choice of one mode versus the other, possibly further narrowed by activation scope (in Puppeteer: `DirectorOnly`, `CastOnly`, `Company`). (*Verification: §6.1 and §6.6; primary code anchor `Reaction.cs:19–23` and `Reaction.cs:67–80`*).
12. **The artifact’s elision action integrates declarative trajectory correlation with journal density.** When the pattern correlates a multi-event trajectory (an order’s `Initiate` → `Confirm` → `Pay` sequence, for instance), the runtime can mark the constituent events as elided in the journal. Subsequent rehydrations skip past them: the trajectory is closed,

its component events have no further bearing on any decision. This closes a conceptual loop with Paper 1 — the *skips* introduced there as a journal-density mechanism become, in the present construct, the natural outcome of a declarative correlation rather than a post-hoc optimization. In Puppeteer this is realized as `Metadata.Elide()` on the Reaction’s metadata plane (§6.5; primary code anchor `Reaction.cs:715-832` and `DiaryStorage.EventElisionStorage.MarkEventsAsElided`).

When NOT to use this approach

The primitive of pragmatic partition is general; the implementation pattern that realizes it has narrower limits. We name three regimes in which the analysis presented here either does not apply, applies only partially, or invites overreach.

A. Domains where every effect must be transactionally atomic with the response

Some domains — banking core ledgers, certain regulated transaction systems — require that every observable effect of a verb commit in lockstep with the response. The pragmatic partition cannot be exercised here without violating the domain’s own contract. The opt-in framing offered in claim 7 does not soften this: opt-in is a license the developer extends to a feature; if the feature has no license to extend, the option is null.

B. Teams without appetite to think in terms of partition decisions

The primitive shifts a design responsibility onto the developer that some teams do not want. A team that treats Reactions as “fire and forget” — without modeling the contract that follows — loses the very property the primitive was supposed to confer. The framework can lower the friction; it cannot supply the design judgment.

C. Workflows where the deferred branch’s failure is structurally inadmissible

A pragmatic partition presupposes that the deferred branch can fail or retry without compromising the verb’s already-committed contract. Workflows in which a downstream failure of the deferred work must abort the verb retroactively — sagas with compensating actions are one such pattern, but the right place to discuss them at depth is the companion paper on Choreography. Some such workflows are admissible under the partition; others are not. The honest position is that the partition is not the right primitive for every long-running workflow.

1. Introduction: a developer says things

This paper makes a design theory contribution. It identifies a structural primitive that frameworks for actor-native systems have left implicit — the developer-controlled partition of an event’s implied work into a “now” branch and a “deferred” branch with a placement decision attached — and derives the design principles required to exercise that primitive at scale. The instantiation that demonstrates the construct is realizable is the Reaction primitive of the Puppeteer framework, treated in §6. The genre is the one Hevner, March, Park, and Ram (2004) name design science research: empirical evidence is presented in the form of a working artifact rather than a controlled experiment.

A developer is modeling a purchase-order endpoint. They are not thinking about transactions, consistency contracts, or CQRS sides. They are thinking about roles and what each role does next: *“the cashier confirms the purchase and locks the requested items. Then a confirmation goes out, and the rewarder evaluates whether any promotional campaign applies.”* They say it at a whiteboard, and a few hours later it is in code — not as a set of architectural decisions translated through a vocabulary, but as the same sentence the developer just spoke. In C# a developer says things — defines classes, writes methods, encodes invariants. What an actor-native runtime adds is that the same act of saying things now extends to the endpoint level: who handles the request, what the response promises, what is handled afterwards.

That extension carries a question the developer has not been asked to answer in industrial practice for some time, and most of this paper is about it. When the developer said *“first this, then that, then the other”*, they drew a partition — between what the verb’s response promises *now* and what is *deferred and placed elsewhere*. They drew it without naming it, on pragmatic grounds: latency, parallelization, and the sense that some concerns simply do not belong inside a domain method. The literature has names for the consequences of that partition — domain libraries that close, fast verbs by construction, opt-in eventual consistency — but the developer never reached for those names. They reached for a sentence about who does what, and the runtime was kind enough to let them write the sentence as the program.

The classical actor model — Akka, Orleans, Proto.Actor and their lineage — provides actor isolation and message passing, but it does not provide a declarative, journal-respected mechanism for what the verb commits to as part of its contract. What happens after the verb is, in those frameworks, side effect: a method call into another grain, a `tell` to another actor, a registered timer, a write to a queue. None of those are part of the actor’s program in the sense the journal preserves (substrate-level *program* throughout this paper refers to the construct of Paper 2 §1.2: the pair of domain library and journal of invocations). This paper argues that Reactions are that missing mechanism — and that without it, the journal-density property established in Paper 1 and Paper 2 cannot survive contact with operational requirements that span the verb’s response.

This paper takes that sentence apart in turn. §2 names the partition the developer drew. §3 examines the artifacts the developer composed to write it: a verb on the actor — a command, a query, or a check-then-command — and the Reactions that handle what the verb deferred. §4 looks at what makes writing a Reaction cheap or expensive, and why the answer matters more than it appears at first. §5 follows what falls out when the writing is cheap: three consequences, ranked. §6 grounds the discussion in code from the public Puppeteer codebase. §7 places the design against Akka and Orleans, §8 addresses the predictable objections, §9 sets the work in the broader literature, and §10 returns to the developer’s sentence and observes what changed.

2. The pragmatic partition: now versus deferred

2.1 The first cut is *who*

The first cut the developer makes when modeling is not “what feature is this” — it is *who*. Cashier. Packer. Deliver. Rewarder. Once the actors are named, the work distributes itself: each role naturally absorbs the verbs that belong to it, and the partition between *what I do now* and *what I see go by and react to afterwards* emerges from the modeling, not from a separate engineering decision. The developer never says “this is opt-in eventual consistency”; they say what each role does. That sentence carries both role and feature — the cashier role *and* the priority order, what is settled before responding versus what is handled after — and it is not feature-by-feature; it is role-by-role-with-feature.

2.2 The partition is drawn, not deduced

Formally, the situation is sharper. Every domain effect implied by the event sits, by formal correctness alone, in *now*: nothing in the verb’s contract entitles the developer to defer anything. The license to defer is therefore not deduced — it is **drawn**. The developer draws it while modeling, on pragmatic grounds, and the grounds are three. First, a latency budget the verb cannot exceed without breaking the actor’s serial-mailbox invariant — the topic of §5.2. Second, an opportunity to parallelize work across roles or across nodes — the placement question of §2.3. Third, a categorical sense that certain concerns (a confirmation going out, an analytics ping, a third-party update) do not belong inside a domain method at all, regardless of timing — the case developed in §3.3. The three grounds are independent of one another, and a single deferral may rest on more than one.

2.3 Placement is part of the same act

[*Skeleton — to iterate in chat. Refined 2026-05-06 to anchor the axis on the two canonical points (Cue and Job) per claim 11.*] At the same act of drawing the now/deferred partition, the developer also decides where the deferred work runs. The axis is not continuous: it has two canonical points named explicitly by the framework. *Cue* runs immediately on a separate thread, against the actor’s

live in-memory state; *Job* runs on demand against the replicated journal, freely placeable on a follower in another pod or another machine. Section 6.1 develops the two modes in detail. The point relevant here is that partition and placement are one decision, not two — choosing what gets deferred and choosing which of the two structurally distinct modes carries it are the same act, not separable concerns.

This partition — its drawing and its placement, taken as one act — is the primitive of the rest of the paper. Everything that follows is downstream of it. §3 examines the artifacts through which the developer writes the partition (a verb on the actor and the Reactions that handle what the verb deferred), naming the two purposes those Reactions serve. §4 looks at what makes writing them cheap or expensive. §5 follows what falls out when the writing is cheap.

3. The double purpose of Reactions

[*Skeleton.*] Before naming what Reactions exclude from the domain library, the paper must name what they do **not** exclude. The domain library is not tool-free. It is built under the legitimate programming paradigm — currently object orientation — that sustains the language of the domain itself. Once that legitimate tool is named (§3.1), the two reasons to defer can be developed without inviting the misreading that “pure” means “tool-free”: pragmatic deferral (§3.2) and categorical segregation of operational pollution (§3.3).

3.1 The legitimate tool inside the domain

The rich library is written in a host language under a programming paradigm — currently object orientation. Classes, inheritance, polymorphism, invariants, domain methods, and the composition of conceptual primitives are the *legitimate tool* of the domain. They live inside the library; they are what the library is made of. The categorical segregation developed in §3.3 does not exclude this paradigm — on the contrary, it is what protects it.

The DSL of an actor runtime does not compete with this paradigm. It is not a second implementation of the domain; it is the language by which the domain is invoked. The relation is structurally analogous to that between SQL and a relational engine: SQL does not duplicate the engine’s logic; it names operations the engine performs. A short SQL statement can trigger joins, locks, and execution plans far heavier than its surface text. The DSL of an actor runtime carries the same kind of leverage — a few lines invoke domain methods whose implementation lives once in the library and whose behavior may be deep. Two endpoints that share invocation lines are not duplicating knowledge; they are composing distinct calls over the same library, in the sense Hunt and Thomas (1999) named when they defined DRY as the prohibition on duplicate *representation of knowledge*, not on the appearance of similar text. The polymorphism of the domain is a first-class citizen of the DSL (Paper 1 covers the parser and symbol-table treatment of inheritance and interfaces).

A second observation belongs here, anticipating §5.1: the closability argument that follows is about the boundary of the library, not about its exhaustiveness. The library can keep growing in domain concepts under its legitimate paradigm — new classes, new invariants, new relations — and that growth is healthy and expected. Practical incompleteness is not a defect of the closability argument. What closes is the absorption of *operational* tools that change with the ecosystem; what remains open is the domain’s own conceptual development.

3.2 Pragmatic deferral

The first reason a developer defers work is operational. The verb of the actor must complete promptly: an actor sustains its target throughput when its mailbox advances quickly, and a verb that takes too long delays every subsequent verb behind it. The constraint is not that I/O must be absent from the verb — formally permitted, an I/O may sit inside a **PerformCommand** — but that I/O accumulates badly. Many verbs each making a network call, or one verb whose latency is unbounded, both collapse the throughput the actor model is supposed to sustain. The fast verb is structural rather than aspirational, and the developer protects it by deferring.

Three patterns, each at a different scope, sustain the fast verb. *I/O hoisting at the caller* — the caller resolves expensive lookups and passes the resolved values into the **PerformCommand**, so the actor receives data and never fetches. *Saga-phasing between events* — when a workflow genuinely requires interleaved external calls, the actor advances one phase, releases, an external coordinator does the I/O, and the response re-enters as the next event. *Reactions* — work that the verb commits to as part of the contract but does not block on, deferred to a **Cue** or **Job** (§6.1). The three are not alternatives; they compose. What §3.2 names is the third — the case where the deferral is owned by the actor itself, declared as a **Reaction**.

A practical recommendation belongs to the third pattern. A **Reaction**’s body is fast-verb code, with the same throughput constraint as any verb on the actor: it is invoked exactly once per match. That guarantee — *exactly once per match* — is what permits a **Reaction** to declare a side effect (sending a Kafka message, calling an external service) inside what is otherwise pure domain code. Were the same side effect to live inside a **PerformCommand**, it would violate the actor principle that commands compute against in-memory state without external I/O; lifting it into a **Reaction** keeps the actor’s command path pure while keeping the side effect’s logic in the domain layer where it belongs.

The same property — *exactly once per match* — is what allows the pattern to survive multi-datacenter replication, but only when the side effect is mediated by a broker rather than performed directly against shared infrastructure. Under journal replication, what travels between datacenters is the action: its **actionId** and parameters. Each datacenter runs the same domain code on its own actor, and each datacenter’s actor runs the same **Reaction** in turn, against its own

locally-replicated journal. If the Reaction emits to its *local* broker, and a *local* consumer downstream writes to a *local* RDBMS or BI store, every datacenter behaves identically and independently — the topology is symmetric. If instead the Reaction performs a synchronous write to a shared external RDBMS, both datacenters now contend for the same external resource on every match, and the operational independence the architecture was buying is lost.

The recommended pattern is therefore to keep slow I/O *outside* the Reaction body altogether: the Reaction emits a message to a broker — Kafka in the canonical case — and a consumer downstream (a BI sink, an analytics writer, a third-party adapter) performs whatever heavy I/O the requirement demands. A typical BI consumer reduces to one insert and several selects against its own store; the actor never sees that load. The framework permits direct RDBMS calls inside a Reaction; the recommendation is not to use them.

3.3 Categorical segregation: encapsulation, not exclusion

Operational concerns demanded by requirements but extrinsic to the domain — email, BI, third-party messaging, telemetry, webhooks — land in Reactions, not in domain methods. The crucial point: the segregation is of *propagation*, not of *presence*. A Reaction can live inside the same library as the domain methods it observes — declared in the same project, next to the classes it cares about. What the framework requires is that the Reaction be its own artifact, with its own scope of effects, rather than collapsed into the body of a `PerformCommand`. The structure is therefore one of co-location without co-mingling. A Reaction whose purpose is to send a confirmation email will, of course, contain a call to an email service; the technology lives there, encapsulated, near the domain methods it serves but distinct from them. What the segregation prevents is that the email concern leak into a domain method that ought to know nothing about email — and that the next domain verb the actor processes inherit a dependency on the email API by virtue of being in the same class. *Reactions are not technology-free; they are the encapsulation that keeps technology from propagating into the domain.*

The function is structurally analogous to an anti-corruption layer in DDD, but architectural rather than stylistic: any complete instantiation treats the deferral artifact as a first-class citizen of the language, and the developer’s defaults route operational concerns there. In Puppeteer this artifact is the Reaction. The legitimate paradigmatic tool inside the library (§3.1) is preserved precisely because the encapsulation is non-leaky.

The non-leak is structural, not stylistic. The Reaction’s emit path runs as a query: `PerformEmit` is invoked with `isQuery:true`, which blocks the script from using `expose` or declaring globals, takes a read lock that runs in parallel with queries and other emits, and leaves the journal untouched (§6.5; verification at `ActorHandler.cs:1662-1733`). The technology of the Reaction (a Kafka producer, an HTTP webhook, a BI sink) executes; the journal does not record

it as a domain event; the actor’s state does not absorb the call. The language refuses to let the operational concern leak back into the actor in the parse phase, before the developer ever has the chance to make a mistake about it.

Section 3.3 connects directly to Paper 1: the anti-porosity transversal, treated there as domain + endpoint + persistence, extends here to a fourth layer — the operational edge of the domain. *Reactions are the anti-porous boundary at the operational edge.*

3.4 Orthogonality of purpose and placement

The two purposes named in §3.2 and §3.3 — pragmatic deferral and categorical segregation — and the two placement modes named in §6.1 — **Cue** and **Job** — are independent decisions. A pragmatic deferral may place close to the action as a **Cue** near-continuation, or far from it as a **Job** running on a journal-only follower; a categorical segregation may do the same. The cells of the resulting two-by-two are all populated in practice: a confirmation email may be a **Cue** (purpose categorical, placement local) or a **Job** on a follower (purpose categorical, placement remote); a heavy projection update may be a **Cue** against the live actor (purpose pragmatic, placement local) or a **Job** distributed across the cluster (purpose pragmatic, placement remote). The developer chooses both dimensions at the moment of declaring the artifact; the two dimensions remain independent in any complete instantiation — there is no “default placement for categorical concerns” or “obligatory mode for pragmatic deferrals.” Each declaration states its own combination.

A second observation about the partition belongs here. The partition is not only a *pragmatic* decision (latency, parallelization, categorical separation); it is also a *preservation* mechanism. When the actor replicates across datacenters (§3.2), the journal travels with it and so does the code: each datacenter executes the same Reactions on its own actor against its own locally-replicated journal. Were the verb’s consequences side effects bolted onto the actor at runtime — **tells**, registered timers, callbacks — the replication would either lose them or reproduce them incoherently. Because Reactions are part of the program, declared in the DSL and replicated as code, the actor behaves as its program says, in every datacenter, every time it rehydrates.

3.5 Pattern matching against the actor’s trajectory

A Reaction is not an event handler. The framework’s primitive is more general — and more difficult to find an analogue for in adjacent runtimes. A Reaction is a rule the engine applies over the actor’s journal: a sequence of one or more **Seek** stages that, taken together, describe a *trajectory* through the actor’s history. The pattern matches not when one event arrives, but when the actor has, over its lifetime, gone through the stages the rule names. Identifiers captured in one stage propagate to the next, correlating the events: an **order** named in the first **Seek** is the same **order** that must appear in the next, and in the next. The

Reaction is part of the actor’s program — declared in the DSL, parsed at build time against the domain’s libraries, replicated as code together with the journal — not a side effect attached to the actor at runtime.

Three properties are jointly characteristic. *Multi-event*: the smallest match is a saga, not a single message. *Statically compiled*: the pattern is parsed against the domain’s `Libraries` in build phase — `[_:Customer].InitiateOrder(order:Order)` resolves to the class `Customer` and to the signature of `InitiateOrder`; if either does not exist in the domain, the Reaction fails to construct. *Trajectorial, not payload-based*: what the matcher inspects is the sequence of *invocations* the actor recorded — its conduct — not the body of any one message. These three together distinguish the primitive from event handlers in Akka, grain reminders or streams in Orleans, and topic consumers in Kafka. The closest analogue in the broader literature is complex event processing, but CEP engines typically operate on external streams without the host language’s type system available; a Reaction operates inside the actor’s own type universe.

This is the property that licenses everything else. Without trajectorial matching, “*the cashier’s verb is fast and a Reaction handles the rewarder afterwards*” reduces to a callback. With it, what the developer has written is “*when, in the cashier’s history, an order was initiated, then confirmed, then paid, react*” — a declaration about how the actor’s lifecycle unfolds, not about what to do when a single event arrives. The matching machinery is examined in §6.2; a worked example with the order lifecycle is in §6.3.

3.6 Reactions are journal-indexed, not event-driven

A Reaction is not an event handler. It is a journal-indexed program cursor.

The structural property is this: **a Reaction traverses the journal monotonically and exactly once**. No journal entry is ever processed twice by the same Reaction — not after a crash, not after a restart, not after a replica hand-off. Reprocessing is not something the Reaction body avoids; it is something the construct forbids by shape.

The property does not rest on deduplication logic, idempotency wrappers, or guards written inside the Reaction. It is upstream of all of those. A Reaction’s position over the journal — one position per `Seek` of the pattern — is itself part of the actor’s persistent state, advances only forward, and is validated on load such that any non-monotonic checkpoint is refused. The cursor is the Reaction’s identity over time; loss of the cursor is loss of the Reaction.

The runtime’s commit discipline exists to sustain this property, not to add it. The cursor’s advance is durable before the Reaction’s action takes effect, and a process failure between those two moments leaves the cursor ahead, never behind. The entry has already been past the cursor by definition; on restart the Reaction’s scan begins from a position that excludes it. The asymmetry is

deliberate: re-executing on the same entry is the failure mode the construct is designed to forbid, while failing to execute is the failure mode it is designed to expose — surfaced as a recoverability event rather than as silent re-execution.

The reader who is about to encounter cross-actor causation in Paper 4 should hold this property fixed. A **Causation.Continue** invocation that records a cross-actor send in the sender’s journal cannot fire twice against the same journal entry — not as a runtime accident, but as a consequence of what a Reaction is. Tell turns the journal into the boundary of causation precisely because the journal is a once-traversed program, not a replayable log of handlers. The property is established here and depended on there.

4. Friction as the architectural guardian

The two purposes of §3 share a guardian. If Reactions cost the developer more lines or more cognitive load than inline code, the partition is not exercised. Both purposes collapse simultaneously: verbs degrade *and* the domain library is contaminated by precipitation.

Reactions must be at least as ergonomic as inline code. If deferring costs the developer more than not deferring, the partition isn’t exercised — verbs degrade AND the domain library is contaminated by precipitation. The contamination is the worse failure: it is hard to revert and propagates couplings. The friction is an architectural responsibility of the framework, not a failure of the developer’s discipline.

The contamination is the worse of the two failures, for two reasons. First, a slow verb is observable and gets fixed: monitoring catches it, profiling diagnoses it, the team rewrites the slow verb on a known schedule. A polluted domain method looks correct and does not — its tests pass, its code reads cleanly, the email it sends is the right email. The dependency on the email API is invisible from the outside, hidden behind the method’s signature, until the day the developer needs to revert it. Second, the contamination propagates. Once `sendEmail(...)` lives inside a domain method, every caller of that method inherits the coupling — directly, by transitive call; indirectly, by sharing a class with that method and inheriting its dependencies in test setups, in mocking frameworks, in the surface area of integration. Reverting the design later requires touching every call site, every test, every place the class has appeared as an object of composition.

The architectural responsibility is therefore not to teach the developer discipline. It is to make Reactions, by construction, no more expensive than inline code. The framework’s contribution is not the partition itself — the developer always *could* have deferred, in any actor framework. The contribution is friction reduction along two dimensions. The first is language-level cost: declaring a Reaction is a few lines of fluent DSL, comparable to or shorter than an inline implementation of the same effect (§6.1, §6.5). The second is textual proximity: Reactions can be written next to the endpoint they observe — in the same file,

a few lines below the `PerformCommand` they continue — so that the developer reading the endpoint sees, on the same screen, what else happens around it: this confirmation goes out, that loyalty rule fires, this analytics counter is updated. The Reaction’s location is itself a documentation aid. When the cost of doing the right thing exceeds the cost of doing the wrong thing, no amount of design review will hold; when the costs are equal or inverted — and when the right thing is also the more legible thing — the partition exercises itself. The framework that takes this seriously is the framework that gets the partition for free.

5. Three consequences (ranked)

[Skeleton. The three consequences fall out of the primitive (§3) when its exercise is low-friction (§4). They are ranked, with closability primary, per the signed thesis of 2026-05-05. Subsections 5.1, 5.2, 5.3 in the order of importance argued by the rhetoric of the series.]

5.1 A domain that closes

The primary consequence of the partition, when its exercise is low-friction, is a domain library that closes. The library is not just kept clean of operational pollution at any one moment; it is structurally protected against the absorption of operational tools that arrive in the future. Email today, Slack tomorrow, a webhook protocol the year after — none of these find their way into a domain method, because the path of entry has been removed at the language level (§3.3). The library that the team is writing now will still be the library the team is reading in three years, and the conceptual primitives it contains will still mean what they meant when they were declared.

At some point a domain must be declarable as “task done”. A library whose definition must absorb every present and future operational tool — email today, Slack tomorrow, a webhook protocol the year after — is a library that never closes. The only way it closes is if it stays pure: free of operational concerns by construction. Reactions are the artifact that makes that closure possible.

Two clarifications matter. First, what closes is the boundary, not the conceptual growth of the domain itself. The library can keep absorbing new domain concepts under its legitimate paradigm — new classes, new invariants, new relations — and that growth is healthy and expected. Practical incompleteness is not a defect of the closability argument:

If Puppeteer is not exhaustive, that is a practical limitation, not a claim that the domain must be seen as complete.

Second, “task done” is an organisational sentence, not a software-engineering one. A team that can declare its domain library complete with respect to operational tools can build everything else — refactor, onboarding, upgrade,

extension — on a stable foundation. In most enterprises today, “the domain” does not exist as a library at all. It exists as classes scattered across each csproj, copied between services, drifting independently over years until “the customer model” or “the order model” has become a dozen incompatible structures across the codebase. The team speaks of these as if they were one thing; in operational reality they are not. A library that closes its boundary to operational tools while remaining open to conceptual growth is the structural condition under which a single domain artifact, maintained as one over years, becomes economically possible at all.

The closability argument extends the anti-porosity transversal of Paper 1 to a fourth layer. Domain libraries shaped for the operations they describe and the conceptual primitives they admit, not for the operational tools requirements happen to demand this year, are libraries that the team can put down and pick back up without forgetting what they meant.

5.2 Fast verbs by construction

Verbs in single-digit milliseconds are not a postulate of this framework, but their explanation is layered. The actor’s verbs are fast first because the actor’s state lives in memory: `PerformCmd` and `PerformQry` operate on it without touching disk or external stores. This is claim 1 of this paper — logical CQRS — and the broader argument of Paper 1. They are fast also because the domain methods invoked from a verb are themselves rich but local: a single verb can dispatch many calls into the host language and across many levels of the domain library, each operation cheap because each operates on the same in-memory state (Paper 2, in the empirical measurements of the purchase example). The Reaction’s contribution is third in this layering, not first: it preserves the speed of the verb against the requirements that would otherwise force I/O into it.

The structural mechanism is, in any instantiation that exercises the partition, that the deferral artifact runs outside the verb’s write semaphore. In Puppeteer, a Reaction does not run inside the write semaphore of `PerformCommand`; it runs after the command has persisted, on a separate thread (in the case of `Cue`) or on demand against the journal (in the case of `Job`). The verb returns to the caller without waiting for any deferred step to begin. The actor’s serial-mailbox invariant — which would otherwise impose a throughput cap whenever a verb does anything slow — becomes a throughput floor: peak throughput is what verbs are *able* to achieve once the slow work has been hoisted out by I/O hoisting at the caller, by saga-phasing between events, or by Reactions on the deferred branch.

What §5.2 names, then, is the third layer of the explanation. The first two — in-memory state, rich local methods — are already present in any actor framework that respects them. What this paper adds is the condition under which they survive contact with operational requirements: a Reaction primitive cheap enough to absorb every effect that would otherwise be tempted to live inside the verb.

Section 6 develops the mechanism in code references; the structural commitment is summarised in `Reaction.cs:715-832` and `ActorHandler.cs:1662-1733`.

5.3 Opt-in eventual consistency

The third consequence honors the forward-reference from Paper 1 §6: the eventual consistency that classical CQRS treats as a defining commitment becomes, in actor-native systems, a shadow of the partition rather than its design choice.

The classical model is opt-out. Read and write stores are physically separated; their synchronisation is asynchronous by default; the application accepts the contract, including its observability gap, as the price of the architecture. Greg Young’s *CQRS Documents* make the trade-off explicit: stronger contracts are recoverable, but only by application-level effort layered over the framework’s defaults.

The actor-native model is opt-in. The actor itself is the read model — `PerformCmd` and `PerformQry` operate on a single in-memory state (claim 1) — so within the actor’s boundary, reads and writes are strongly consistent without any ceremony. The eventual-consistency contract appears only at the deferred branch: when the developer declares a `Reaction`, what happens after the verb’s response is named explicitly. That act of naming is what the contract is. Each deferred effect is documented at the point of deferral, with a name and a placement, and the system makes no further claim about its observability than what the developer has stated.

This inversion is structural, not stylistic. A reader of the system can list the eventual-consistency windows of any given verb by reading the `Reactions` declared near it; the runtime offers nothing else. A reader of a classical CQRS system needs to know, separately, the topology of the read store, the synchronisation policy, the lag profile under load, and the application’s strategies for compensating against them. The Puppeteer reader has, in the worst case, the source of the `Reaction`; the classical reader has, in the best case, an architecture diagram and a runbook.

6. Implementation in the Puppeteer framework

[Skeleton — reorganized 2026-05-06 to reflect the technical handoff verified against `Puppeteer/EventSourcing/Follower/` and `ActorHandler.cs`. Path:line citations have been verified against the current Puppeteer mainline.]

6.1 The two modes: Cue and Job

The `ReactionMode` enum exposes two values, `Cue` and `Job`, declared in `Reaction.cs:19-23`. The two are not points on a continuum chosen by the developer at runtime; they are structurally distinct execution modes, picked when the `Reaction` is defined and binding the rest of its semantics.

`Cue()` is the live-actor mode. The framework registers the Reaction via `actorHandler.AddRecordWrittenCallback(...)` after an initial catch-up batch (`Reaction.cs:563-568`); from that point on, every record written to the journal wakes the Reaction’s push loop with the new entry. The loop runs on a separate thread and sleeps adaptively with backoff between empty checks. Because the Reaction shares the actor’s in-memory state with the director — the same rehydration, no separate automaton — its access is sub-second and inexpensive while the actor is hot. `ReadForward()` is mandatory: a `Cue` cannot read backward, since its purpose is to react to the actor’s ongoing trajectory, not to its past.

`Job()` is the journal-driven mode. There is no push loop. The Reaction runs only when an external scheduler or a call to `Reactions.Execute()` triggers it. It reads the journal directly rather than the actor’s memory, which makes the Reaction *portable*: any process with read access to the replicated journal can host the work. The case enabling specialised workers — *“this pod runs the followers for campaigns; that pod runs the heavier analytics followers; another pod activates only periodically to handle older events”* — is built on this primitive. Both `ReadForward()` and `ReadBackward()` are admissible.

`Job` does not rehydrate the actor. When the Reaction needs domain-computed data, the canonical mechanism is `expose` (§6.7) — the command pre-writes the data to the journal alongside the action, and the Reaction reads it during its own rehydration without ever waking the actor. This keeps `Job` cheap to host on a journal-only follower; reaching into the live actor would defeat the cost model that justifies the mode in the first place. The non-rehydration boundary is therefore not a missing feature but a structural commitment that preserves the distinction between the two modes.

The two modes correspond to two structurally distinct points on the proximity/urgency axis introduced in claim 11. A comparative table:

Aspect	<code>Cue</code>	<code>Job</code>
State source	Actor’s live memory (same process)	Rehydration from journal
Lifts a separate automaton	No	Yes (follower hydrates)
Trigger	<code>OnRecordWritten</code> push callback	Explicit invocation
Latency	Sub-second	Batch / on-demand
Distributable	No	Yes
Direction	<code>ReadForward</code> only	Forward or backward

The choice between them is an architectural decision, not a tuning knob. A Reaction modeled as `Cue` cannot transparently become a `Job`: the two assume different things about what is in memory, who hosts the work, and whether

the actor must be live for the Reaction to make progress. This is the §2.3 axis materialised — partition and placement are one act because, at the moment of partitioning, the developer is also choosing which of these two modes will carry the deferred work. Both modes operate as journal-indexed cursors per §3.6: the choice between them governs *who walks the journal and when*, not whether the cursor exists.

6.2 Static pattern matching against the domain libraries

The Reaction’s binding to the domain is established at build time, not at run-time. Three layers of validation make this concrete.

The first layer is static parsing in `Pattern.cs`. Every reference of the form `[_:Class]`, `[var:Class]`, or `Class(param:Type, ...)` is resolved against the actor’s `Libraries`, case-insensitively. If the class or the constructor signature does not exist in the domain library, the Reaction fails to construct — there is no runtime path that can recover from a missing type. The `Libraries` here are the same domain types that Paper 1 named as the legitimate tool inside the library and that Paper 2 treated as the substrate against which DSL programs name their operations; the Reaction is one more language layer that compiles against that substrate.

The second layer runs per event during rehydration. `Reaction.SolveActionReferences()` (`Reaction.cs:997`) parses the script of each `ActionEventData` exactly once and caches it under an LRU of 100 entries; identifiers that the pattern captured are marked `IsParameter=true` so subsequent stages can reuse them without re-parsing. The cache amortises the cost of parsing the same script many times across many events; the LRU bound prevents memory growth in workloads with high script diversity.

The third layer is the runtime walk. `MatchTree.TryMatchAtLevel()` traverses the tree of stages by BFS (`WithSharedHydration`, where captures from earlier stages remain visible to later ones) or by DFS (`WithIndependentHydration`, where each branch carries its own hydration context). The choice between the two is part of the Reaction’s definition and shapes how identifier propagation behaves across `Seek`, `ThenSeek`, and `ThenFinalSeek` stages.

The parser, importantly, does not execute the script’s expressions. It only knows the types. Matches over non-literal values are therefore by structure and type, not by value equality — with two exceptions: literals (a constant in the pattern) and `Id` parameters with `IsParameter==true` (the captured identifiers of §6.3). The composition rules round out the layer: a Reaction has at least one `Seek` and any number of `ThenSeek` stages, optionally terminated by a `ThenFinalSeek`; each `Seek` may carry several `OnMatch` clauses, all of which must match against the same underlying script. The build-time validation stops a Reaction from being defined against a domain it does not fit; the runtime walk only ever evaluates patterns the `Libraries` have already approved.

6.3 Multi-event patterns: trajectory, not event

The simplest non-trivial Reaction names three stages of an order’s life and correlates them by both the order instance and a primitive identifier:

```
actor.Reactions
  .DefineReaction("OrderCompleted")
  .Job().DirectorOnly().ReadForward().WithSharedHydration()
  .Seek("Initiate")
    .OnMatch("[_:Customer].InitiateOrder(order:Order, number:int)")
  .ThenSeek("Confirm")
    .OnMatch("order.Confirm(number)")
  .ThenFinalSeek("Pay")
    .OnMatch("order.Pay(number, [_:PaymentMethod]")
  .Program.Emit("...");
```

Read aloud, the pattern says: “*when, in the actor’s history, a customer initiated an order with a given number N , and that same order with that same number N was later confirmed and paid — react.*”

Five details of this small example carry the weight of the primitive.

Identifier propagation. The captured `order` named at `Seek("Initiate")` is the same `order` *instance* that must appear at `ThenSeek("Confirm")` and at `ThenFinalSeek("Pay")`; the captured `number` is the same primitive *value*. The matcher correlates by typed identity for object captures and by literal equality for primitive captures — both kinds of identifiers, once captured, are treated as fixed for the remainder of the trajectory. A `Confirm` invoked with a different number, or for a different order instance, would not advance the trajectory; the trajectory is locked to *this* order with *this* number once the first stage matches.

Non-contiguity. Between `Confirm` and `Pay` the actor may have processed any number of unrelated commands; the matcher steps over them. The trajectory is a path through the journal, not a contiguous window.

One match per stage. Each `Seek` and `ThenSeek` matches exactly once. For one-to-many correlations — an order with several items being added before payment — the framework uses `RepeatSeek`, which is restricted to the initial position of the pattern. The surrounding language and its limits are documented in §6.4.

Pattern over conduct. What the matcher inspects is the sequence of invocations the actor recorded — its conduct as a sequence of `PerformCommand` calls — not the body of any one message. The Reaction sees what the actor *did*, not what arrived in its mailbox.

Static binding. `Customer`, `Order`, `PaymentMethod` are types from the actor’s domain library, validated at build time against `Libraries`; the symbol table of the actor is available to the matcher, so polymorphism and type relations are respected without further configuration.

Three properties together — *trajectory*, *static binding*, *typed propagation* — make this primitive qualitatively distinct from event handlers, grain reminders, and CEP rules over external streams. The first two §3.5 already named; the third is what makes the example above readable as a saga rather than as three loosely-coupled callbacks.

6.4 Filters, time, quantifiers

The matcher accepts three families of refinement on top of the base pattern.

Where clauses. `Where(expression)` (`ReactionEngine.cs:47-60`) chains a boolean condition onto a `Seek`. Inside the expression the developer has access to `@Now` (the event's timestamp, non-nullable), `@EntryId` (the journal entry's identifier, non-nullable), the identifiers captured by the pattern, and the global `time` instance of the framework's `Temporal` type. `time.Days(n)`, `time.Hours(n)`, `time.Minutes(n)` and so on (`Reaction.cs:525`) construct `TimeSpan` values for relative windows. The framework rejects always-false comparisons at build time: comparing `@Now` or `@EntryId` against `null` raises a `LanguageException`, since their types do not admit a null sentinel.

Quantification. The base pattern matches each `Seek` exactly once; for one-to-many correlations the framework provides `RepeatSeek(name)` (`Reaction.cs:318-329`), which accumulates multiple matches at a single level before transitioning to the next stage. The chained modifiers (`ReactionEngine.cs:117-149`) are `.GroupBy(varName)` (group accumulated matches by the value of a captured variable), `.Accumulate(name)` (expose the accumulated matches under a name visible to the next stage), `.Distinct(varName)` (filter duplicates by a captured variable), and `.Until(...)` taking either an integer count or a `TimeSpan`. The constraint is positional and intentional: `RepeatSeek` is admissible only at the start of the pattern. There is no `ThenRepeatSeek` mid-chain — a topology of *one event*, *then several*, *then one* must either model the repetition first or decompose into two cooperating Reactions.

Lifecycle. `SetExpirationDate(DateTime)` (`Reaction.cs:619-623`) shuts down the Reaction wholesale after a date; `IsExpired` (`Reaction.cs:627-634`) returns `true` once `DateTime.UtcNow` exceeds the expiration. `SetActive(bool)` (`Reaction.cs:613-617`) is the manual override. These two differ in scope from `Until(timeout)`: `Until` closes a single accumulation window inside a `RepeatSeek`; `SetExpirationDate` and `SetActive` toggle the entire Reaction. The two scopes are useful for different problems — `Until` lives in the pattern; `SetExpirationDate` lives outside it.

6.5 The three action planes

A Reaction takes exactly one action when its pattern matches. That action is expressed through one of three named *planes*, each describing a different surface of the system the matched trajectory is permitted to affect.

Plane	What the verb touches	What it cannot touch
Program	the actor's domain libraries, read-only	the journal, the actor's state, any global
Causation	the actor's own journal (a journaled cross-actor send)	the domain libraries or the metadata plane
Metadata	the journal's elision register	the domain state or any external effect

A Reaction is therefore not an unbounded scripting surface attached to a pattern. It is a constrained declaration of which surface of the system the matched trajectory is permitted to affect. The builder enforces that only one plane can be configured per Reaction; configuring a second plane after one has already been set is a construction error. The *exactly-one-action* rule is structural, not conventional — the developer cannot accidentally route a single match into two different planes because the surface refuses the second configuration call at build time.

Program — read-only execution against the domain libraries The **Program** plane executes a script under the same regime as a query. The parser runs in query mode, the script is forbidden from declaring globals, the **expose** keyword is rejected, the journal is not written, and the runtime takes a read lock that runs in parallel with other queries and emits.

The script may invoke arbitrary external technology — an HTTP client, a Kafka producer, a SignalR hub, a BI sink. What the language denies is any path by which those calls could feed back into the actor's state. This is not a discipline rule kept by convention; it is rejected at parse time, before the developer has the chance to write the leak. A Reaction may observe the actor and affect the outside world, but it cannot feed that effect back into the actor.

The optional **when:** guard performs a second read-only check at firing time:

- the pattern asserts that *this trajectory occurred in history*,
- the guard asserts that *it still makes sense to react now*.

The distinction matters under replay, catch-up after downtime, and replica rehydration — the three situations in which the temporal distance between history and the present can be arbitrary. The pattern is bound to the trajectory; the guard is bound to the clock.

A worked example. Consider a Reaction that, after a confirmed purchase, pushes a real-time toast notification to the customer's UI:

```
seller.Reactions.DefineReaction("NotifyPurchaseToast")
    .Cue().Company().ReadForward()
    .Seek("Purchase")
```

```

        .OnMatch("[s:Seller].purchase($orderId, $date, $amount, $customer)")
    .Program.Emit(
        "notifier.Push(@customer, @orderId);",
        when: "check (notifier.IsFresh(@date)) WARNING 'toast stale, drop';"
    );

```

The domain verb `Seller.purchase(...)` carries no reference to the notification hub, to UI presence, to toast timing, or to any operational concern of the notification layer. All of that lives inside the Reaction: the call to `notifier.Push(...)` invokes the extrinsic technology, and the guard decides at firing time whether the toast still has an audience. Under the steady path the push fires within seconds of the journal entry; under catch-up or replay, the same match arrives at the matcher hours or days later, the guard returns false, and the emit is suppressed. The freshness policy is plain code in a library class — `ToastNotifier` — that the actor sees through its `Libraries` but that the domain verb never references. The segregation is guaranteed because there is no syntactic route by which this Reaction could import the notifier back into `Seller.purchase`.

Causation — journaled cross-actor continuation The Causation plane is the journaled cross-actor send. A Reaction on this plane writes exactly one sentence into the sender’s own journal instructing another actor; the sender records the causation, the receiver processes the message independently through its own endpoint. Cross-actor coordination therefore appears in the program as a domain fact rather than as infrastructure — neither a saga step nor a tracing span nor a choreography event, but a sentence in the actor’s history.

This plane is developed in detail in Paper 4, which introduces the primitive, names the structural conditions under which it preserves cross-actor program continuity, and presents the comparative case study against sagas, choreography, and distributed tracing. Here it is named only as the second of the three surfaces a Reaction may touch.

Metadata — declaring a trajectory closed The Metadata plane marks the events covered by the pattern as elided. After a trajectory has been correlated, its component events no longer participate in any future decision; subsequent rehydrations walk past them.

This is the conceptual bridge to the *skips* introduced in Paper 1. What appeared there as a journal-density optimisation is here the natural outcome of declarative correlation. The developer is not cleaning the journal after the fact. The developer is declaring “*this trajectory is closed; its component events no longer decide anything.*” The skip is the consequence of that declaration, not its cause.

Parameter injection `WithParameters(...)` allows captures from the pattern matching to be modified or augmented before the action runs, regardless of which plane is configured. The parameters are pre-populated with the pattern’s

captures, so the idiomatic use is to enrich them rather than to construct them from scratch.

Why the planes matter Without named planes, a Reaction would be an unbounded script runner attached to history — a feature whose semantics would depend on which calls the developer happened to write. With named planes, a Reaction becomes a one-sentence statement about which layer of the system is allowed to change when a historical pattern is recognised. The same primitive that detects the trajectory also declares the surface of effect, and the surface of effect is selected from a closed set of three rather than from the open set of all calls a script could make.

That restriction is what makes the segregation between

- domain verbs,
- cross-actor causation,
- operational side-effects, and
- journal maintenance

reliable across teams and across time. The reliability is not a property of the developer’s discipline. It is a property of the surface the language permits the developer to address.

6.6 Activation: where the Reaction runs

After `Cue()` or `Job()`, the developer chooses an activation scope (`Reaction.cs:67–80`). `DirectorOnly()` runs the Reaction only on the primary replica — appropriate when the work must be unique across the topology, such as orchestrations that should not be duplicated. `CastOnly()` runs only on followers (secondary replicas), enabling the specialised-workers case named in §6.1: followers that hydrate against the journal alone, hosting `Job` Reactions that the director does not need to evaluate. `Company()` runs on the director and on all followers; appropriate when each replica should hold its own evaluation of the trajectory, perhaps for local cache invalidation or for redundant emit paths.

Activation composes with the `StageManager` — the topology director of the `Choreography` module. The `StageManager` decides who is director and who is follower at any moment; the Reaction inherits that identity to decide whether to fire. A Reaction defined `CastOnly` becomes silent on the director and active on whatever process the `StageManager` has elected as a follower; a director-to-follower handoff therefore changes the population of running Reactions without any further action by the developer. The deployment-time mechanism that keeps such handoffs safe is `Theater.aliveGate` (claim 8); it is treated in detail in Paper 5.

6.7 `expose` versus `Program.Emit`

The two mechanisms are easy to confuse and are not the same thing. They are addressed in the same direction — feeding downstream systems data the actor has computed, without forcing those systems to rehydrate the actor — but from opposite sides of the journal.

`expose` is a keyword of the actor’s language, valid only inside a `PerformCommand`. When the verb runs, the script can mark a value as exposed, and the framework persists it into the `ExposeData` of the resulting journal entry alongside the `Action`. It is the *writer’s* externalisation point: the cashier confirming an order can expose the order number, the calculated total, the resolved campaign — and downstream systems can consume those values directly from the journal without having to walk the actor’s state.

`Program.Emit` is the Reaction’s *reader’s* externalisation point. A Reaction running over the journal — whether `Cue` or `Job` — has access to the `ExposeData` of the events it traverses (`Reaction.cs:573, includeExposeData=true`), and it can use those values in its `Where` clauses, in its captures, and in the parameters of its emit. What it cannot do is *produce* an `expose`: `PerformEmit` blocks the keyword by parsing with `isQuery:true` (§6.5). The asymmetry is structural and intentional. `expose` is the writer’s hook into the journal; `Program.Emit` is the reader’s hook out of it.

6.8 What does not exist — honest limits

A technical paper that reports what a primitive supports without reporting what it does not is incomplete. The Reaction primitive does not, today, provide:

- **Negative or absence patterns.** No `Without`, `NotMatch`, `Negate`, `Unless`, `Missing`. “*X happened but Y did not happen afterwards*” is not directly expressible. The conventional workaround is a periodic `Job` that matches `X` and consults the actor’s state via `Program.Emit` with a `when`: check to confirm `Y` has not occurred.
- **Traditional regex-style quantifiers.** No `AtLeast(n)`, `AtMost(n)`, `Exactly(n)`, `OneOrMore`, `Optional`. The only quantification primitive is `RepeatSeek` with `Until(count)` or `Until(timeout)`, restricted to initial position.
- **Absolute time windows** as a dedicated method (`Within(start, end)`). Expressed instead via `Where("@Now >= ... && @Now <= ...")`.
- **Optional `ThenSeek`.** A `ThenSeek` cannot be marked optional. Optional branches are written as separate Reactions.
- **Job-mode rehydration of the actor.** A `Job` Reaction reads only the journal; it does not wake or rehydrate the actor itself. When domain-computed state is needed by the Reaction, the canonical mechanism is `expose` (§6.7), which pre-writes the data alongside the command. Silent rehydration inside `Job` is intentionally absent — collapsing this boundary would erase the structural distinction between `Cue` and `Job` (§6.1) and

reintroduce the cost the design was avoiding.

- **Cross-actor dispatch with journal-recorded causation — historically a gap, now resolved.** When v0.1 of this paper was published, this was a recognized limitation: cross-actor causation was mediated by `ITransport` infrastructure outside the sender’s journal, leaving the broker as the seam. Since v0.1, the framework has gained the third action plane — `Causation.Continue` (the Tell primitive) — that records the cross-actor dispatch as a sentence in the sender’s own journal while leaving the transport choice to the developer’s `ITransport` and the target actor’s processing entirely to its own endpoint. The construct, the design rationale, and the comparison against sagas, choreography, and distributed tracing are the subject of Paper 4. The bullet is preserved here as historical record of how the gap was named before being filled; the present paper does not develop the primitive (it is referenced from §6.5 as the third plane).

These absences are honest design boundaries of the present implementation. Some are gaps in the original sense — pending features that may arrive — and some, like the last one, are deliberate commitments without which the surrounding primitive would lose coherence. The framework’s scope is what the framework supports; what it does not support is what the developer must build over it or model differently.

7. Comparison with Akka and Orleans

7.1 Akka actors and Become/Receive

Akka’s actors model behavior change with `Become` and `Unbecome`, swapping the receive partial function as the actor moves between modes. A booking actor might `Become(opened)`, then `Become(confirmed)`, then `Become(paid)`; each receive defines a different set of messages the actor accepts and how it handles them. The partition this expresses is across *actor states* — what the actor will accept next changes with the state — and the developer’s mental model is a state machine the actor walks through.

Reactions express a different partition. The actor’s `PerformCommand` is one verb, not many — it does not `Become` between calls — and the partition that Reactions add is across the *event boundary*: when a command has been processed and persisted, what happens afterwards is the Reaction’s domain. Akka’s mechanism is suited to actors whose interface changes as state evolves; Puppeteer’s mechanism is suited to actors whose interface is stable but whose effects after each verb are rich. Neither subsumes the other; they answer different questions.

7.2 Orleans grains, timers, and reminders

Orleans’ grains support timers (in-process, not durable across grain reactivation) and reminders (durable, scheduled). Both share the placement intent of `Job`: deferred work, possibly on another silo, scheduled rather than co-located. The

differentiator is friction. Registering an Orleans timer or reminder is procedural — a method call, a callback signature, a registration step in the grain’s activation lifecycle — and the work itself is general C# code. A Reaction is a few lines of fluent DSL declared next to the verb that triggers it (§4), with a static pattern that compiles against the domain library (§6.2) and a constrained set of actions (§6.5). The framework’s contribution is not a feature Orleans lacks but a friction profile Orleans does not aim for; the developer who wants the partition exercised by default in Orleans will write more code than the developer who wants the same in Puppeteer.

7.3 Where the design spaces converge and diverge

Akka, Orleans, and Puppeteer address the same structural pressures: the actor’s serial mailbox, the cost of state hydration, the cost of distributing work across nodes, the difficulty of keeping fault isolation honest. The three answer those pressures with different design commitments. Akka commits to a flexible behavior model in which the actor’s interface itself can evolve; Orleans commits to virtual actors that materialise on demand, with placement and scheduling as runtime concerns. Puppeteer commits to an externalised DSL with a homoiconic journal (Paper 2) and an anti-porous domain (Paper 1) — and Reactions are the language layer of that commitment: declarative pattern matching over the actor’s trajectory (§3.5), with a strict separation between the actor’s command path and what runs afterwards (§5.2). The Puppeteer position is one point in a non-trivial design space, not its peak.

Nothing in the present paper claims Reactions are the only realization of the construct. An Akka or Orleans extension that exposed a declarative DSL for trajectory matching with build-time binding to the host language’s types would satisfy the same construct; this paper presents one realization, not the realization.

8. Counter-arguments

8.1 “Eventual consistency is a known anti-pattern in business systems”

The objection treats eventual consistency as a uniform property of the system, taken or rejected wholesale, and reads the warning literature on it as applying everywhere it appears. The framing is correct for *opt-out* eventual consistency, the kind classical CQRS introduces by separating read and write stores: every read is potentially stale, every write is potentially unobserved, and the application has to recover stronger contracts wherever the business demands them. Under that framing, the warning is well-earned.

The model developed in this paper inverts the framing (§5.3). Reads and writes inside the actor are strongly consistent — `PerformCmd` and `PerformQry` operate on the same in-memory state. Eventual consistency appears only at the deferred

branch, named by the developer one deferral artifact at a time, with a placement attached. There is no system-wide consistency contract that the application must compensate for; there is, instead, a list of named windows the developer has chosen to admit. The literature warning applies to the framework that imposes eventual consistency as a default; a reader of an instantiation that exercises this construct can answer “what is eventually consistent here?” by reading the deferral artifacts declared near the verb and nothing else.

The honest concession (claim 6’s 20%): some business domains require that every observable effect of a verb commit in lockstep with the response — banking core ledgers, certain regulated transaction systems. The opt-in framing does not soften this; in those domains, the developer cannot extend a license that does not exist, and the partition’s primitive is not the right primitive (§When NOT, A). The objection lands where it lands; it does not generalise to every business system that adopts the framework.

8.2 “Reactions are async events with a different name”

The objection holds only if Reactions are read at their surface — as something that runs after a verb, on a different thread, with a callback shape. At that surface they look like async events, and the objection follows.

Read at the structural level of the construct, the artifact realizing the partition is something else. An async event handler responds to a single message; the construct’s matching primitive matches a *trajectory* — a sequence of one or more stages naming a saga, a lifecycle, or an anomaly through the actor’s history (claim 10, §6.3). The pattern is parsed against the domain’s libraries at build time, not against text at runtime: a reference to a class the domain does not contain fails to construct (§6.2). Identifiers captured in one stage are typed and propagated to the next, which is what correlates the events into a story — an async event handler has no analogue. The placement axis with two canonical points (in Puppeteer’s instantiation, *Cue* and *Job*) carries activation scopes (*DirectorOnly* / *CastOnly* / *Company*) that compose with the StageManager’s topology (§6.6); an async event has none of that surface. The action plane choices (*Metadata.Elide*, read-only *Program.Emit* with optional *when:* check, and the journaled cross-actor *Causation.Continue*) are constrained primitives — particularly the read-only *Program.Emit* whose underlying *PerformEmit* structurally blocks *expose* and journal writes (§6.5) — designed to enforce the categorical segregation of §3.3 in the parse phase, not at runtime.

The honest concession (claim 6’s 20%): the surface mechanism — work happens after the verb returns, on a separate thread or on demand — is shared with async event systems. A reader who looks only at the trigger sees something familiar. The argument of this paper is that the trigger is the least interesting part of the primitive.

8.3 “Even with low-friction Reactions, the developer can pollute the domain”

The objection is correct in spirit: friction reduction is not a substitute for discipline. A determined developer can put `sendEmail(...)` inside a `PerformCommand` even when a Reaction one line below would do the job; the framework cannot prevent it. Were the argument that friction reduction *replaces* developer judgment, the objection would land.

The argument is different. Friction reduction is the lever the framework can pull; developer judgment is the lever it cannot. When friction is high — when the right thing costs more than the wrong thing — no amount of training, code review, or architectural pep-talk holds at scale. When friction is low, training and review have something to lean on; the developer who chooses inline anyway is doing so against an obvious cheaper alternative, and the choice becomes legible to reviewers as a deliberate departure rather than a path of least resistance.

The honest concession is at the seam where friction reduction reaches its limit: a careless team will write polluted code under any framework, and a disciplined team will write clean code under any framework. The framework’s contribution is the *gradient* between these — making clean code the default, pollution the exception, and the cost of getting it wrong recoverable rather than irreversible. The objection becomes an argument for friction reduction, not against it: in a world where developer discipline is unreliable, the only thing the architecture can do is shape the cost surface so that the path of least resistance is also the path of least damage.

8.4 “DSL scripts repeat lines across endpoints — this violates DRY”

The objection assumes DRY as a textual prohibition. In its original formulation (Hunt & Thomas, 1999), DRY prohibits the duplicate representation of *knowledge*, not the appearance of similar lines: “*every piece of knowledge must have a single, unambiguous, authoritative representation within a system.*” The domain library lives once, in OOP; DSL scripts are the language in which an actor is invoked, structurally analogous to SQL. Two SQL queries that share JOIN patterns are not in violation of DRY — they are compositions over a shared schema. The same is true of two DSL endpoints that share invocation lines: the knowledge is in the library that the lines invoke, not in the lines themselves.

Concession (the 20%): if two DSL endpoints share *domain logic* — not invocation patterns but actual computation — that logic should refactor into a method of the library. The line is between invocation and implementation, not between unique and repeated text.

8.5 “Reactions should be able to rehydrate the actor for richer state queries”

The choice between modes is the structural distinction itself — full state queries belong to the live-actor mode, not to the journal-driven one (§6.1). When the data is anticipated by the developer, the journal pre-write mechanism (**expose** in Puppeteer, §6.7) puts the data where the deferral artifact will consume it during its own rehydration, without ever waking the actor. The journal-only-without-rehydration boundary is what makes the journal-driven mode cheap to host on a remote follower; lifting it would collapse the two modes into one fuzzy mode and reintroduce the cost the original design was trying to avoid.

Concession: there are rare cases of data that no one anticipated would be needed by a future Reaction, and that already exist in many past events without an **expose**. The workaround is operational — add **expose** now, run a migration that replays old events to write the **expose** retroactively, or build an auxiliary **Job** that materialises a side projection. Costly, but solvable, and rare enough that it does not justify reopening the primitive.

9. Related work

9.1 Actor Model and CQRS foundations

The actor model originates with Hewitt (1973), which introduces the actor as the universal unit of computation: an entity with private state, a mailbox for incoming messages, and a single thread of behavior responding to them. The model has accumulated decades of refinement — Agha (1986), Hewitt (2010), Bonér et al.’s *Reactive Manifesto* (2014) — but the core remains: state isolated, computation message-driven, concurrency through the multiplication of actors rather than the sharing of state. Puppeteer’s actor sits in this lineage, with the addition that the journal of received messages is itself the persisted artifact rather than a derived log.

Command Query Responsibility Segregation (CQRS) is named in Young (2010), with intellectual antecedents in Meyer’s command-query separation (1988) and Fowler’s reflections on enterprise architecture (2002). The classical formulation separates write and read stores at the level of the system architecture, with eventual consistency between them as the design contract. This paper engages CQRS not as the primary frame but as the literature that names the consequences of the partition this paper develops; the “logical CQRS” of claim 1 is the classical pattern read at a different level — one in-memory state, two verbs over it, the eventual-consistency contract appearing only at the deferred branch (§5.3, §8.1).

9.2 DDD and the anti-pattern literature

Domain-driven design originates with Evans (2003), where the *anemic domain* and *smart UI* antipatterns are diagnosed: domain types reduced to data con-

ainers with behavior emigrated to service layers, and business rules pulled into the presentation/transport edge. Vernon (2013) catalogs the implementational pitfalls of DDD-by-vocabulary-only — bounded contexts and aggregates named but never structurally enforced. Fowler (2003, in *AnemicDomainModel*) gives the antipattern its most cited articulation. These three sources name the industrial reality §5.1 of this paper describes: in most enterprises, “the domain” does not exist as a library — it exists as scattered classes whose names suggest cohesion and whose substance does not.

Fowler’s *Patterns of Enterprise Application Architecture* (2002) is the relevant catalog for the DTO antipattern: a Data Transfer Object intended for transport that becomes load-bearing in the domain, contaminating both. The same volume names the *Distributed Object* antipattern — the chatter problem this paper alludes to in §3.1 when it argues for the SQL-analogous use of a DSL as invocation rather than transcription.

Hunt and Thomas (1999) define DRY in *The Pragmatic Programmer* as the prohibition of duplicate *representation of knowledge*, not of repeated text — the formulation §3.1 and §8.4 recover against a more recent industrial reading. Sandi Metz’s *AHA — Avoid Hasty Abstractions* (2016) provides the natural complement: the cure for repeated text is sometimes more text, not the wrong abstraction.

9.3 CEP and adjacent runtimes

Complex event processing has a substantial literature — Luckham’s *The Power of Events* (2002), Etzion and Niblett’s *Event Processing in Action* (2010) — concerned with detecting patterns over streams of events, often from heterogeneous sources, often without the type system of the consuming application available. The pattern matching machinery of §6 shares structural intent with CEP rules but operates inside the actor’s own type universe: patterns compile against the domain’s **Libraries** at build time (§6.2), captured identifiers are typed and propagated (§6.3), and the events being matched are the actor’s own conduct rather than external streams. The closest analogue from CEP is the typed event pattern of engines that integrate with strongly-typed host languages; the operative difference is that those engines treat the pattern as a runtime artifact, while Puppeteer’s pattern is a compile-time artifact that fails to construct against a domain it does not fit.

Akka and Orleans are the most relevant adjacent runtimes (§7). Their documentation is the canonical reference for Akka’s **Become/Receive** and Orleans’ grain timers, reminders, and stream APIs — the comparators against which §7 contrasts the Reaction primitive without claiming superiority.

9.4 Prior papers in this series

This paper builds on two prior contributions. *Anti-porous architecture: a unified design principle for CQRS + Actor + Event-Sourcing systems* (Paper 1) names

the structural defect — porosity — that the present paper extends to a fourth layer (the operational edge of the domain, §3.3) and the legitimate tool — the domain library shaped by its operations — that §3.1 carries forward. *Program-value separability: the structural precondition for compilation, caching, and dense journaling in a DSL runtime* (Paper 2) names the structural property of DSL programs that makes their compilation, caching, and dense journaling possible at all; the static binding of Reaction patterns against the domain’s **Libraries** (§6.2) is one more layer of language built atop that substrate.

Preserving semantic continuity across actors: a tell-based approach without orchestration (Paper 4) extends the work-partition primitive across the actor boundary. Where the present paper develops **Program.Emit** and **Metadata.Elide** as in-actor verbs of the Reaction’s action planes, Paper 4 develops **Causation.Continue** — the Tell primitive — as the journaled cross-actor verb that closes the historical limit named in §6.8 of this paper’s v0.1. The two papers compose: Reactions name the developer’s partition of an event’s implied work into now and deferred (the present paper); Tell names how the deferred branch can address another actor while remaining a sentence in the sender’s program. *Zero-downtime deployment via journal-based state handoff* (Paper 5, forthcoming) treats the deployment-time mechanism (**Theater.aliveGate**) referenced in claim 8 and §6.6.

The combined retrieval graph — Paper 1, Paper 2, Paper 3, Paper 4 — converges on a runtime in which every layer (domain, endpoint, persistence, operational edge, language) is shaped by what the operations of the domain require, and on a primitive of work-partition (this paper’s contribution) that admits eventual consistency only as an opt-in shadow of the developer’s choice — extended across actor boundaries by Paper 4.

10. Conclusion: a developer says things — and now also at the endpoint

In §1 a developer was modeling a purchase-order endpoint by saying things. *“The cashier confirms the purchase and locks the requested items. Then a confirmation goes out, and the rewarder evaluates whether any promotional campaign applies.”* That sentence was the program before it was code. Each phrase in it carries one of the technical names this paper has used.

- *“The cashier confirms the purchase and locks the requested items”* — a verb on a single in-memory state shared by reads and writes; the literature calls this *logical CQRS* (claim 1).
- *“Then a confirmation goes out, and the rewarder evaluates...”* — the partition between *now* and *deferred*, with placement chosen at the moment of partitioning (claims 3 and 5). Each phrase after the first is itself a Reaction declared next to the verb.
- *“A confirmation”* lands in a Reaction rather than in a domain method because the email concern does not belong inside the verb the cashier ex-

poses — categorical segregation (claim 4b), encapsulating the technology where it can be exercised without propagating into the rest of the domain.

- “*The cashier, the order, the rewarder*” — these live inside a domain library that closes its operational boundary while continuing to grow conceptually under its legitimate paradigm (claims 7 and 9).

The developer reached none of these technical names while modeling. They reached for a sentence about who does what. The names are downstream — useful for reasoning about the system, for defending the design, for situating it against Akka and Orleans and the broader CQRS literature. The sentence is upstream — useful for getting the work done.

At some point a domain must be declarable as “task done”. A library whose definition must absorb every present and future operational tool — email today, Slack tomorrow, a webhook protocol the year after — is a library that never closes. The only way it closes is if it stays pure: free of operational concerns by construction. Reactions are the artifact that makes that closure possible.

In C# a developer says things. In the DSL of an actor-native runtime, a developer says things too — about endpoints, about who does what, about what gets handled afterwards. The act is the same; the surface is bigger. What is new is not how the developer thinks. It is what the language lets them say.

Appendix A. Code references

All references are to the Puppeteer codebase. Path:line citations were verified against the mainline at the time of writing (2026-05-06). Releases after this date may shift line numbers; the symbolic references (method names, class names) are stable and locate the same artifact regardless of line drift.

Reaction primitive — Puppeteer/EventSourcing/Follower/Reaction.cs

Range	What it shows	Cited in
19-23	<code>ReactionMode</code> enum (<code>Cue</code> , <code>Job</code>)	§6.1
67-80	Activation builders (<code>DirectorOnly</code> , <code>CastOnly</code> , <code>Company</code>)	§6.6
318-329	<code>RepeatSeek</code> declaration with positional restriction	§6.4
525	<code>time</code> global registered as <code>Temporal</code> instance	§6.4

Range	What it shows	Cited in
563-568	Cue mode push-callback registration via	§6.1
573	AddRecordWrittenCallback includeExposeData=true during rehydration	§6.7
613-617	SetActive(bool) manual override	§6.4
619-623	SetExpirationDate(DateTime) shutdown	§6.4
627-634	IsExpired predicate	§6.4
656-658	The three action planes exposed as properties (Program , Causation , Metadata)	§6.5
708-713	WithParameters parameter modifier	§6.5
692-698	SetMetadataAction() internal hook for Metadata.Elide()	§6.5
700-706	EnsureNoActionConfigured() build-time guard (exactly-one-action rule)	§6.5
715-832	Action handlers (ExecuteAction , ExecuteProgram , ExecuteCausation)	§5.2, §6.5
848	MarkEventsAsElided invocation (Metadata plane)	§6.5
769	PerformEmit invocation from ExecuteProgram	§6.5
997	SolveActionReferences() per-event resolution	§6.2

Reaction action planes — `Puppeteer/EventSourcing/Follower/Planes.cs`

Range	What it shows	Cited in
11-19	Header comment naming the three planes (Program , Causation , Metadata) and their verbs	§6.5

Range	What it shows	Cited in
44	<code>Program.Emit(string script)</code> — read-only emit	§6.5
53	<code>Program.Emit(string script, string when)</code> — read-only emit with check	§6.5
76	<code>Causation.Continue(string script)</code> — journaled cross-actor verb (Tell)	§6.5, Paper 4
97	<code>Metadata.Elide()</code> — mark matched entries as elided	§6.5

Reaction language layer — `Puppeteer/EventSourcing/Follower/ReactionEngine.cs`

Range	What it shows	Cited in
47-60	<code>Where(expression)</code> clause registration with build-time validation	§6.4
117-149	<code>RepeatSeek</code> modifiers (<code>GroupBy</code> , <code>Accumulate</code> , <code>Distinct</code> , <code>Until</code>)	§6.4

Read-only emit path — `Puppeteer/EventSourcing/ActorHandler.cs`

Range	What it shows	Cited in
1662-1733	<code>PerformEmit</code> method (read-only, <code>isQuery:true</code> , read lock)	§3.3, §5.2, §6.5, §6.7
1659	In-source comment: <i>“Parser: isQuery:true — bloquea expose (que persistiria al journal) y declaracion de variables globales”</i>	§6.5
1677	<code>parser.Parse(isQuery: true, isCheck: false)</code>	§6.5

Range	What it shows	Cited in
1712	<code>rwLock.EnterReadLock()</code> taken before <code>Perform</code>	§6.5

Other modules (referenced without line numbers)

File	What it shows	Cited in
Puppeteer/EventSourcing/Fullpwnet/Pattern.cs	pattern descriptions against Libraries	§6.2
Puppeteer/EventSourcing/Fullpwnet/MatchTree.cs	BFS/DFS hydration walk	§6.2
Puppeteer/EventSourcing/Fullpwnet/MatchTree.cs	storage interface	§6.5

Code provenance

Source-code references in this paper resolve against the public Puppeteer repository at commit `2f31f96` (2026-05-18). The snapshot is archived in Software Heritage under the following persistent identifier:

```
swh:1:dir:10e7e6bad7eb77b6c2e406762026177f95c3ae92;
  origin=https://github.com/alvaronCubo/puppeteer;
  anchor=swh:1:rev:2f31f9674a5de816bdf1bf9d8360ff218a02e4da
```

Inline references of the form `file.cs:NN` (e.g., `ActorHandler.cs:38`) resolve against this snapshot. A reader can construct a per-file SWHID by adding the qualifiers `;path=<path>;lines=<NN>` to the directory SWHID above. Future commits to the repository may renumber lines; the SWHID preserves the cited state independently of any future change to the repository or its hosting.

Acknowledgments

The author used large language models (including Claude and ChatGPT) as editorial assistants for language refinement, structural feedback, and literature navigation. All original ideas, terminology, theoretical constructs, and technical content presented in this work are solely the author’s.

Appendix B. Bibliography

Online sources accessed at the moment of publication; dates filled in at release.

- Agha, G. (1986). *Actors: A Model of Concurrent Computation in Distributed Systems*. MIT Press. ISBN 978-0-262-01092-7.
 - Akka documentation. <https://akka.io/docs/> (accessed 2026-05-09).
 - Bonér, J., Farley, D., Kuhn, R., Thompson, M. (2014). *The Reactive Manifesto, v2.0*. <https://www.reactivemanifesto.org/>
 - Etzion, O., Niblett, P. (2010). *Event Processing in Action*. Manning Publications. ISBN 978-1-935-18221-4.
 - Evans, E. (2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley. ISBN 978-0-321-12521-7.
 - Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley. ISBN 978-0-321-12742-6.
 - Fowler, M. (2003). *AnemicDomainModel*. <https://martinfowler.com/bliki/AnemicDomainModel.html> (accessed 2026-05-09).
 - Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). *Design science in information systems research*. MIS Quarterly, 28(1), 75–105.
 - Hewitt, C. (1973). *A Universal Modular Actor Formalism for Artificial Intelligence*. Proceedings of the 3rd International Joint Conference on Artificial Intelligence (IJCAI-73), pp. 235-245.
 - Hewitt, C. (2010). *Actor Model of Computation: Scalable Robust Information Systems*. arXiv:1008.1459. <https://arxiv.org/abs/1008.1459>
 - Hunt, A., Thomas, D. (1999). *The Pragmatic Programmer: From Journeyman to Master*. Addison-Wesley. ISBN 978-0-201-61622-4.
 - Luckham, D. (2002). *The Power of Events: An Introduction to Complex Event Processing in Distributed Enterprise Systems*. Addison-Wesley. ISBN 978-0-201-72789-1.
 - Metz, S. (2016). *The Wrong Abstraction*. <https://sandimetz.com/blog/2016/1/20/the-wrong-abstraction> (accessed 2026-05-09).
 - Meyer, B. (1988). *Object-Oriented Software Construction*. Prentice Hall. ISBN 978-0-13-629031-1.
 - Microsoft Orleans documentation. <https://learn.microsoft.com/en-us/dotnet/orleans/> (accessed 2026-05-09).
 - Vernon, V. (2013). *Implementing Domain-Driven Design*. Addison-Wesley. ISBN 978-0-321-83457-7.
 - Young, G. (2010). *CQRS Documents*. https://cQRS.files.wordpress.com/2010/11/cQRS_documents.pdf
-
- Rivera, A. (2026). *Anti-porous architecture: a unified design principle for CQRS + Actor + Event-Sourcing systems*. Paper 1 of this series. 01-anti-porosity.md
 - Rivera, A. (2026). *Program-value separability: the structural precondition for compilation, caching, and dense journaling in a DSL runtime*. Paper 2 of this series. 02-program-value-separability.md
 - Rivera, A. (2026). *Preserving semantic continuity across actors: a tell-based approach without orchestration*. Paper 4 of this series. 04-cross-actor-continuity.md